UK2ME ONLINE

Milestone 3 — Work Plan

Manual weight pricing, complaints, dispatch tracking, virtual wallet, store-fee registry & role management

Project: UK2ME Revamp (monorepo — apps/client · apps/admin · apps/backend) · Prepared: 4 June 2026 · Status: Planning → ready to build

Project: UK2ME Revamp (monorepo — `apps/client`, `apps/admin`, `apps/backend`) **Prepared:** 4 June 2026 **Status:** Planning → ready to build

> **Scope.** Seven items from the latest client notes (**R9–R15**). Two of them **amend Milestone 2**: the refund concept is dropped in favour of **Complaints** (R10), and money-back cases are handled through the new **Virtual Wallet** (R13) instead of gateway refunds. See §3 *Amendments to Milestone 2*.

1. Purpose

Milestone 2 automated invoicing, region baskets, the delivery-date engine and email reliability. Milestone 3 fills the operational gaps that surfaced once that flow was mapped end-to-end:

- Some item **categories have no weight class**, so the weight engine can't price Nigeria postage automatically — staff need a way to quote those **manually** without blocking the order.
- "Refunds" are really **complaints**; actual money-back should become **store credit** the customer can re-spend, not a gateway reversal.
- Customers need to **always see where their order is**, and to receive a **tracking ID** the moment an order is dispatched to Nigeria.
- The **store → UK-warehouse delivery fee** is still entered by hand; it should be a **per-store value managed in admin**.
- The admin needs **role management** — promote/demote staff without touching the database.

2. Requirements (this milestone)

#	REQUIREMENT	SOURCE
R9	**Manual weight pricing ("Request Weight Price").** When an item's category has **no weight class / weight price**, the customer sees a **Request Weight Price** button instead of an instant quote. It notifies the backend + emails admins ("get the weight price for these items and resolve it"). Once an admin sets the manual delivery price, the customer can pay for that item.	This message
R10	**Complaints replace refunds.** Remove the refund concept. A customer raises a **complaint** on an order; admin reviews and resolves/rejects; emails at each step. *(Money-back is handled by R13.)*	This message
R11	**Always-visible delivery status.** The order status is **always shown** to the customer (starting at **Placed**) so they know where their items are at every point.	This message
R12	**Dispatch tracking.** When an order is **dispatched to Nigeria**, the admin enters a **Tracking ID** and **Delivery Company**; the status changes and the customer is **emailed the tracking ID when the status changes**.	This message
R13	**Virtual Wallet.** If a customer has paid and an item goes **out of stock**, the money is returned to their **virtual wallet** as credit, which they can spend on another item.	This message

#	REQUIREMENT	SOURCE
R14	**Store → warehouse delivery-fee registry.** The delivery fee **from each store to our UK warehouse** is configured **per store in admin** (automated into invoice math). Seeded from the client's current list; new stores added via admin.	This message
R15	**Role management engine.** Manage staff roles in admin — **elevate and demote** a user's role — with proper permission gating.	This message

3. Amendments to Milestone 2

These change items already written into the M2 plan (M2 is not yet built, so this is a plan change, not a code rollback):

- **M2 R7 "Order Dispute & Refund" → superseded.** The dispute mechanic stays but is renamed **Complaint** (R10) and the **refund half is removed**: drop `OrderStatus.REFUNDED`, `PaymentStatus.REFUNDED`, and the `requestedRefund` / `refundAmount` fields from the M2 `Dispute` model. Any genuine money-back is issued as **wallet credit** (R13).
- **M2 §6 email matrix** loses the "refund processed" rows and gains the complaint, weight-price, dispatch-tracking and wallet rows in §6 below.

(The M2 doc is left intact as the historical record; say the word if you'd like it edited in place too.)

4. Current state vs. gap

AREA	ALREADY IN CODE	GAP TO CLOSE
Weight pricing	`lib/weight.ts` resolves weight via 5 fallbacks ending in a silent **500 g absolute fallback**; `Category` + `WeightReference` hold the data	Detect "no real weight" (`source: 'fallback'` / category flagged) and switch to **manual request** instead of mis-pricing (R9)
Roles	`lib/auth.ts` `requireAdmin()` exists, but `User` **has no role field**	Add `Role` to `User`; admin UI to elevate/demote; gate endpoints by role (R15)
Tracking	`Shipment { carrier, trackingNumber, status }` model exists but is unused in the flow	Wire dispatch → create `Shipment` (company + tracking) → status change → **email with tracking** (R12)
Stores	`AdapterState { name, domain, region, enabled }` already lists stores (for scraping)	Add a **store→warehouse fee** per store, admin-managed, used by invoice math (R14)
Money-back	M2 planned **gateway refunds**	Replace with **Virtual Wallet** credit + spend (R13)
Disputes	M2 planned `Dispute` + refund	**Complaint** model, no refund fields (R10)
Status visibility	`Order.status` + `OrderEvent[]` exist; client view limited	Always-on **status timeline** for the customer (R11)

5. Data model changes (`apps/backend/prisma/schema.prisma`)

5.1 Roles (R15)

```
enum Role { CUSTOMER  STAFF  ADMIN  SUPER_ADMIN }

User += role Role @default(CUSTOMER)
     +  roleUpdatedAt DateTime?
     +  roleUpdatedById String?
```

`requireAdmin()` and a new `requireRole(min)` read `User.role`. Only `SUPER_ADMIN` (and optionally `ADMIN`) may change roles; guard against **self-demotion** and **removing the last admin**. Every change writes an audit `OrderEvent`-style record (or a small `AdminAudit`).

5.2 Manual weight pricing (R9)

```
Category += requiresManualWeight Boolean @default(false)   // flag categories with no weight class

enum WeightPriceStatus { AUTO  REQUESTED  PRICED }

OrderItem += weightStatus WeightPriceStatus @default(AUTO)
          +  manualDeliveryPrice Float?      // admin-set Nigeria-postage price for this item
          +  manualPriceCurrency String?

model WeightPriceRequest {
  id            String   @id @default(cuid())
  orderId       String
  orderItemId   String?
  productUrl     String
  category       String?
  status         WeightPriceStatus @default(REQUESTED)
  resolvedPrice  Float?
  currency       String?
  requestedById  String?
  resolvedById   String?
  createdAt      DateTime @default(now())
  resolvedAt     DateTime?
  order          Order    @relation(fields: [orderId], references: [id])
}
```

Trigger: at quote/checkout, if `resolveWeight()` returns `source: 'fallback'` **or** the item's `Category.requiresManualWeight` is true, the item is marked `weightStatus = REQUESTED` and shown the **Request Weight Price** button (no auto Nigeria-postage line). An order with any `REQUESTED` item **cannot be invoiced/paid** until every item is `PRICED`.

> **Open question (§9.1):** whether the customer pays the item now and the delivery price later (a *deposit* model), or the order simply waits until the manual delivery price is set and is then paid in one go. This plan assumes the **single-payment** model.

5.3 Complaints (R10) *(replaces M2 §4.6 Dispute; no refund fields)*

```
enum ComplaintStatus { OPEN  UNDER_REVIEW  RESOLVED  REJECTED }

model Complaint {
  id             String   @id @default(cuid())
  orderId        String
  userId         String?
  reason         String          // category + free text
  detail         String?
  status         ComplaintStatus @default(OPEN)
  resolutionNote String?
```

```
  resolvedById   String?
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
  resolvedAt     DateTime?
  order          Order    @relation(fields: [orderId], references: [id])
}
```

5.4 Virtual Wallet (R13)

```
enum WalletTxnType { CREDIT  DEBIT }

model Wallet {
  id        String  @id @default(cuid())
  userId    String  @unique
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  user      User    @relation(fields: [userId], references: [id])
  txns      WalletTransaction[]
}

model WalletTransaction {
  id           String   @id @default(cuid())
  walletId     String
  type         WalletTxnType
  amount       Float
  currency     String          // GBP | USD — credit is currency-specific
  reason       String          // e.g. "out_of_stock:ORDER123"
  orderId      String?
  paymentId    String?
  balanceAfter Float           // running balance for this currency
  createdById  String?
  createdAt    DateTime @default(now())
  wallet       Wallet   @relation(fields: [walletId], references: [id])
}
```

Balances are tracked **per currency** (a £ credit pays UK orders, a $ credit pays US orders). **Out-of-stock:** admin issues a `CREDIT` against the affected order; customer is emailed. **Spend:** at invoice/checkout the customer may apply wallet balance (same currency) — recorded as a `DEBIT` plus a `Payment { provider: "wallet" }`, with the gateway charged only the remainder.

5.5 Dispatch tracking (R12) — reuse the existing `Shipment` model

```
// On the PROCESSING/AWAITING_PURCHASE → SHIPPED transition, admin must supply:
Shipment { carrier = <Delivery Company>, trackingNumber = <Tracking ID>, status = IN_TRANSIT }
// SHIPPED = "dispatched to Nigeria". Email fires on the status change (R5 path) with the
// tracking ID + company. Optional: carrier→tracking-URL templates in AppSetting.
```

No new model needed; we only **require** carrier + tracking when moving to `SHIPPED` and hook the email. *(Open question §9.5: keep the label `SHIPPED` or rename to `DISPATCHED`.)*

5.6 Store → warehouse fee (R14) — extend `AdapterState` (already the per-store registry)

```
AdapterState += storeToWarehouseFee Float?
             +  storeFeeCurrency    String?   // GBP | USD (defaults from region)
```

Admin gets a column/field to set the fee per store; the invoice engine looks up each store group's fee automatically (replacing the hand-typed store postage in M2 §4.3). A store with no fee set surfaces an **admin warning** rather than silently charging £0. Seeded

from the client's current list; new stores added in admin. *(If stores without a scraper adapter are needed, a standalone `Store` table is the alternative — see §9.4.)*

5.7 Always-visible status (R11) — no new model

Driven by `Order.status` (always set, defaulting to `PLACED`) + the `OrderEvent[]` history. The customer order page renders a fixed **status timeline** (Placed → Invoiced → Paid/Processing → Dispatched → Delivered, with Complaint/Cancelled side states) showing the current step and timestamps from `OrderEvent`.

---

6. Email notification matrix (additions)

EVENT	CUSTOMER	ADMIN
Weight price requested	✓ (received)	✓ **(items + links to price)**
Weight price resolved	✓ (now payable)	—
Order dispatched	✓ **(Tracking ID + company)**	—
Complaint opened	✓ (received)	✓ (alert)
Complaint status changed / resolved / rejected	✓	—
Wallet credited (e.g. out of stock)	✓	✓
Wallet used at checkout	✓ (receipt shows credit applied)	—

All routed through the M2 **WS-5 reliable-mailer** path (log + retry + visible status).

---

7. Workstreams

> Each task lists a **verify** check so progress is testable.

WS-10 — Role management engine (R15) *~2 days*

1. Migrate `Role` enum + `User.role` (+ audit fields); backfill existing admins. → *verify: `prisma migrate` clean; current admins keep access.*
2. `requireRole(min)` helper; gate admin endpoints by role. → *verify: a `STAFF` user is blocked from a `SUPER_ADMIN`-only route.*
3. Admin **Users** → **role** UI: elevate/demote, with guards (no self-demotion, keep ≥1 admin). → *verify: promoting a user takes effect on next request; last-admin demotion is refused.*

WS-11 — Manual weight pricing (R9) *~2.5 days*

1. Migrate §5.2 (`Category.requiresManualWeight`, `OrderItem` fields, `WeightPriceRequest`). → *verify: migrate clean; types compile.*
2. Quote/checkout: detect no-weight items (`source:'fallback'` or flagged category) → mark `REQUESTED`, hide auto postage. → *verify: an item in a no-weight category shows Request Weight Price, not a price.*
3. **Request Weight Price** button → creates `WeightPriceRequest`, emails admins with item + link. → *verify: clicking it creates the request and sends the admin email.*
4. Admin **weight-price queue**: set the manual delivery price → item `PRICED`, customer emailed; order becomes invoiceable/payable. → *verify: pricing the item unblocks payment; customer gets the "now payable" email.*

WS-12 — Store → warehouse fee registry (R14) *~1.5 days*

1. Migrate §5.6 fee fields on `AdapterState`; seed from the client's list (when supplied). → *verify: stores list shows editable fees.*

2. Admin UI to set/add store fees; invoice engine pulls the per-store fee automatically. → *verify: changing a store's fee changes the next invoice's store-postage line; a missing fee shows an admin warning.*

WS-13 — Dispatch tracking + status email (R12) *~1.5 days*

1. Admin "Mark dispatched" action requires **Delivery Company + Tracking ID** → writes `Shipment`, moves order to `SHIPPED`. → *verify: dispatch without tracking is refused; with it, a `Shipment` row is created.*
2. Email on the dispatch status change including tracking ID + company (+ optional tracking URL). → *verify: the customer receives the dispatch email with the correct tracking ID.*

WS-14 — Always-visible status timeline (R11) *~1 day*

1. Customer order page renders the fixed status timeline from `Order.status` + `OrderEvent`. → *verify: a placed order shows Placed and lights up later steps as events occur.*
2. Ensure every status transition writes an `OrderEvent` (so the timeline is complete). → *verify: each transition appears with a timestamp.*

WS-15 — Virtual Wallet (R13) *~3 days*

1. Migrate §5.4 (`Wallet`, `WalletTransaction`). → *verify: migrate clean; a wallet is auto-created per user on first use.*
2. **Out-of-stock → credit:** admin action on an order/item issues a `CREDIT` (currency-correct), writes `balanceAfter`, emails the customer. → *verify: crediting raises the customer's balance and sends the email.*
3. **Spend:** checkout/invoice "Apply wallet credit" debits the wallet (same currency) and charges the gateway only the remainder via a `Payment{provider:"wallet"}` + gateway payment. → *verify: a part-wallet payment records both a `DEBIT` and the gateway charge; totals reconcile.*
4. Client **Wallet page**: balance(s) per currency + transaction history. → *verify: page matches the ledger.*

WS-16 — Complaints (R10) *~2 days (replaces M2 WS-8/WS-9 refund parts)*

1. Migrate §5.3 (`Complaint`, `ComplaintStatus`); **remove** the M2 refund enums/fields from the plan's schema. → *verify: migrate clean; no `REFUNDED` states remain.*
2. Client **"Raise a complaint"** on eligible orders → form (reason + detail) → `Complaint(OPEN)` + customer/admin emails. → *verify: submitting creates the complaint and both emails fire.*
3. Admin **Complaints** list/detail → `UNDER_REVIEW` → **Resolve/Reject** with note; each transition emails the customer. → *verify: resolving emails the customer and records the note + timestamp.*

---

8. Sequencing & estimate

```
WS-10 roles ──────────────► (gates the new admin screens)
WS-11 weight pricing ─────► touches checkout + invoice (coordinate with M2 WS-2/WS-3)
WS-12 store fees ─────────► feeds invoice math (M2 WS-2)
WS-13 dispatch tracking ──► uses M2 WS-5 mailer
WS-14 status timeline ────► client order page
WS-15 wallet ─────────────► checkout/invoice payment path (M2 WS-4)
WS-16 complaints ─────────► replaces M2 WS-8; uses M2 WS-5 mailer
```

Indicative effort: WS-10 ~2 · WS-11 ~2.5 · WS-12 ~1.5 · WS-13 ~1.5 · WS-14 ~1 · WS-15 ~3 · WS-16 ~2 → **~13–14 working days**. WS-11, WS-12 and WS-15 should land alongside the M2 invoice/payment work since they share those code paths; WS-10 (roles) is a clean prerequisite for the new admin screens and can start immediately.

---

9. Acceptance criteria (definition of done)

- [ ] Items in a **no-weight category** show **Request Weight Price**; an order with unpriced items **cannot be paid** until an admin sets the manual price; both parties are emailed at request and resolution.

- [ ] **Refunds are gone**; customers raise **complaints**, admins resolve/reject, every step emails the customer.
- [ ] The customer **always sees the order status**, starting at **Placed**, on a timeline that fills in as events occur.
- [ ] Marking an order **dispatched** requires a **Delivery Company + Tracking ID** and emails the customer the tracking ID **on the status change**.
- [ ] An **out-of-stock** item can be returned to the customer's **virtual wallet**; the customer can **spend that credit** on another order; balances and ledger are correct per currency.
- [ ] **Store → warehouse fees** are set **per store in admin** and applied automatically by the invoice engine; missing fees warn rather than charge £0.
- [ ] Admins can **promote/demote** user roles in admin, with guards (no self-demotion, ≥1 admin retained); endpoints are gated by role.

---

10. Open questions (confirm before/early in build)

1. **Weight "deposit" model (R9).** Does the customer pay for the item now and the (manual) delivery price later, or wait and pay once in full after the price is set? *(Plan assumes single payment after resolution.)*
2. **Wallet currency & withdrawal (R13).** Confirm credits are **per-currency** (£ credit → UK orders only, $ → US). Can wallet credit ever be **withdrawn to bank**, or only spent on-site? Does a credit **expire**?
3. **Roles (R15).** Confirm the role set (`CUSTOMER / STAFF / ADMIN / SUPER_ADMIN`?) and **who may change roles** (super-admin only?). What can `STAFF` do vs `ADMIN`?
4. **Store fee shape (R14).** Is the store→warehouse fee **flat per order from that store**, **per item**, or **per kg**? Currency per store/region? Send the **current store list** to seed.
5. **Dispatch label (R12).** Keep status `SHIPPED`, or rename to `DISPATCHED` / "In Delivery"? Do you want per-carrier **tracking-URL** links in the email?
6. **Complaint scope (R10).** Which order statuses can a customer complain about, and what **reason categories** should the form offer?
7. **Out-of-stock trigger (R13).** Admin-only, manual? Is **partial** out-of-stock supported (credit some items, keep the rest), with the order/invoice adjusted?
8. **Status timeline (R11).** Confirm the exact customer-facing **step labels** to display.
